

FULL STACK DEVELOPMENT

FASE 5 | AULA 02 -

QUANDO USAR MICROSSERVIÇOS E MONOLITOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS.....	20
O QUE VOCÊ VIU NESTA AULA?	22
REFERÊNCIAS	23

EXEMPLO

O QUE VEM POR AÍ?

Nesta aula, vamos explorar os dois principais modelos de arquitetura de microserviços: Arquitetura Baseada em APIs e Arquitetura Orientada a Eventos. A arquitetura de microserviços é uma abordagem moderna que facilita a construção de sistemas flexíveis, escaláveis e resilientes. Conhecer as diferentes arquiteturas e componentes envolvidos é essencial para implementar microserviços de forma eficaz.

Primeiro, abordaremos a **Arquitetura Baseada em APIs**, onde cada microserviço expõe uma API bem definida que pode ser consumida por outros serviços ou pela camada de apresentação. Esta arquitetura promove a integração e a reutilização de serviços, permitindo uma comunicação estruturada e consistente entre diferentes partes do sistema. Vamos entender como REST, GraphQL e gRPC funcionam dentro desse contexto, incluindo exemplos práticos de uso.

Em seguida, discutiremos a **Arquitetura Orientada a Eventos**, que utiliza eventos para comunicar e sincronizar os diferentes serviços. Neste modelo, os serviços publicam eventos quando algo relevante acontece, e outros serviços podem reagir a esses eventos de maneira assíncrona. Essa abordagem promove o desacoplamento e a resiliência do sistema, sendo particularmente útil em cenários onde a latência deve ser minimizada e as operações assíncronas são frequentes. Veremos como implementar essa arquitetura usando ferramentas como Apache Kafka e RabbitMQ, com exemplos práticos para ilustrar seu funcionamento.

HANDS ON

Nesta aula prática, apresentaremos os motivadores para escolher a arquitetura adequada para um sistema de microserviços, destacando a importância de desenvolver aplicações que sejam eficientes, escaláveis e adaptáveis a mudanças futuras. Vamos explorar como decidir entre arquiteturas baseadas em APIs e orientadas a eventos, considerando os fatores específicos do projeto, como tamanho do sistema, requisitos de escalabilidade e estrutura da equipe.



SAIBA MAIS

Modelos de arquitetura

Arquitetura baseada em APIs

A arquitetura baseada em APIs é uma das abordagens mais comuns para implementar microserviços. Nesse modelo, cada microserviço expõe uma API bem definida, que pode ser consumida por outros serviços ou pela camada de apresentação. Essa abordagem facilita a integração e a reutilização de serviços, permitindo que diferentes partes do sistema se comuniquem de maneira estruturada e consistente.

API, ou Interface de Programação de Aplicações (Application Programming Interface), é um conjunto de definições e protocolos que permite que diferentes aplicações se comuniquem entre si. As APIs definem os métodos e dados que as pessoas desenvolvedoras podem usar e estabelecem regras sobre como os componentes do software devem interagir. Em essência, uma API serve como um contrato entre o serviço que a fornece e os clientes que a consomem.

Tipos de APIs

REST (Representational State Transfer)

REST é um estilo de arquitetura que utiliza HTTP para fazer chamadas entre sistemas. É conhecido por sua simplicidade e uso amplo.

Características: utiliza operações HTTP padrão (GET, POST, PUT, DELETE), é baseado em recursos e URLs e retorna dados em formato JSON ou XML.

Benefícios: simplicidade, ou seja, fácil de usar e entender, ampla adoção, pois é suportado por praticamente todas as linguagens e frameworks, e flexibilidade, já que pode ser utilizado para uma ampla gama de aplicações.

Desafios: ineficiência para consultas complexas, porque pode ser ineficiente para recuperar dados de múltiplas fontes em uma única chamada, e não tem suporte nativo para operações de streaming e comunicação bidirecional.

Exemplo prático: imagine um serviço de catálogo de produtos em uma aplicação de e-commerce. Para obter informações sobre um produto específico, um cliente pode fazer uma solicitação GET para a URL <https://api.exemplo.com/produtos/123>. O servidor retornará os detalhes do produto no formato JSON.

```
{  
  "id": 123,  
  "nome": "Tênis de Corrida",  
  "preço": 199.99,  
  "estoque": 42  
}
```

Código-fonte 1 – Exemplo 1
Fonte: elaborado pelo autor (2024)

GraphQL

GraphQL é uma linguagem de consulta para APIs e um tempo de execução para realizar essas consultas com seus dados existentes.

Características: permite que os clientes solicitem exatamente os dados de que precisam, nem mais nem menos. As consultas são definidas pelo cliente, não pelo servidor, e há suporte para operações de leitura (queries), escrita (mutations) e assinaturas (subscriptions).

Benefícios: reduz a sobrecarga de dados, pois permite consultas precisas, reduzindo a quantidade de dados transferidos. Tem flexibilidade, se tornando ideal para aplicações com interfaces de usuário complexas, e facilita a evolução de APIs sem quebrar as implementações existentes.

Desafios: curva de aprendizado, já que pode ser mais difícil para aprender e configurar, e complexidade de servidor, pois o servidor GraphQL precisa resolver as consultas, o que pode introduzir ainda mais complexidade.

Exemplo prático: Para obter informações sobre um produto e suas avaliações em uma única solicitação, um cliente pode enviar a seguinte consulta GraphQL:

Quando usar microserviços e monolitos

```
query {  
  produto(id: 123) {  
    nome  
    preço  
    estoque  
    avaliacoes {  
      autor  
      comentario  
      estrelas  
    }  
  }  
}
```

Código-fonte 2 – Exemplo 2
Fonte: elaborado pelo autor (2024)

O servidor retornará apenas os campos solicitados:

```
{  
  "data": {  
    "produto": {  
      "nome": "Tênis de Corrida",  
      "preço": 199.99,  
      "estoque": 42,  
      "avaliacoes": [  
        {  
          "autor": "João",  
          "comentario": "Muito confortável",  
          "estrelas": 5  
        },  
        {  
          "autor": "Maria",  
          "comentario": "Bom custo-benefício",  
          "estrelas": 4  
        }  
      ]  
    }  
  }  
}
```

```
}
```

Código-fonte 3 – Exemplo 3
Fonte: elaborado pelo autor (2024)

gRPC (Google Remote Procedure Call)

gRPC é um framework de chamada de procedimento remoto (RPC) desenvolvido pelo Google e que utiliza HTTP/2 para transporte e Protocol Buffers como a interface de descrição de linguagem.

Características: suporta comunicação bidirecional e streaming de dados, usa Protocol Buffers para serialização eficiente de dados e suporta múltiplas linguagens de programação.

Benefícios: alta performance, sendo ótimo para comunicações de baixo atraso e alto rendimento. É eficiente, pois Protocol Buffers são mais compactos e rápidos que JSON, e tem comunicação bidirecional, com suporte nativo a operações de streaming.

Desafios: mais complexo para configurar e usar em comparação com REST e pode não ser necessário para todas as aplicações, especialmente aquelas com requisitos mais simples.

Exemplo prático: imagine um serviço de chat em tempo real. Um cliente pode abrir um stream gRPC para enviar e receber mensagens de chat. Ele inicia a comunicação com um stub gRPC gerado a partir de um arquivo `.proto` que define o serviço:

```
syntax = "proto3";

service MonitoringService {
    rpc MonitorSensors (stream SensorData) returns (stream
SensorResponse);
}

message SensorData {
    string sensorId = 1;
    float value = 2;
```


Quando usar microserviços e monolitos

```
int64 timestamp = 3;
}

message SensorResponse {
    string status = 1;
    string message = 2;
}
```

Código-fonte 4 – Exemplo 4

Fonte: elaborado pelo autor (2024)

Desta forma, o cliente e o servidor podem enviar e receber dados de sensores de forma assíncrona usando streams bidirecionais.

Estrutura da arquitetura baseada em APIs

A estrutura de uma arquitetura baseada em APIs envolve vários componentes chave, como:

- **Serviços independentes:** cada serviço é desenvolvido de forma independente e é responsável por uma funcionalidade específica. Isso promove a modularidade e facilita a manutenção e a escalabilidade da aplicação.
- **APIs bem definidas:** as APIs servem como contrato de comunicação entre os serviços. Elas são bem definidas, documentadas e versionadas para garantir a compatibilidade e facilitar a integração.
- **API Gateway:** um API Gateway atua como um ponto de entrada centralizado para todas as solicitações que chegam ao sistema. Ele roteia as solicitações para os serviços apropriados, aplica políticas de segurança e pode realizar transformações de dados.

Benefícios da arquitetura baseada em APIs

A arquitetura baseada em APIs **facilita a integração** entre diferentes serviços e a reusabilidade de funcionalidades. As APIs bem definidas e documentadas permitem que os serviços se comuniquem de maneira padronizada.

Outro benefício é que cada serviço pode ser desenvolvido, testado e implantado de forma **independente**. Isso possibilita que diferentes equipes trabalhem simultaneamente em diferentes partes da aplicação sem interferir umas nas outras.

Por fim, **cada serviço pode ser escalado individualmente** conforme necessário. Isso permite uma utilização mais eficiente dos recursos e facilita a adaptação da aplicação a variações na carga de trabalho.

Desafios da arquitetura baseada em APIs

Requer uma **gestão cuidadosa das APIs** para garantir a compatibilidade e evitar a criação de dependências acopladas, incluindo a necessidade de versionamento das APIs e a documentação clara.

Outra desvantagem é que a exposição de APIs pode introduzir **vulnerabilidades** que precisam ser mitigadas com autenticação, autorização e monitoramento adequados. É essencial implementar mecanismos robustos de segurança para proteger os dados e as operações.

Monitorar e depurar uma aplicação baseada em microserviços que se comunicam por APIs pode ser desafiador devido ao número de serviços e à complexidade das interações entre eles. Ferramentas avançadas de monitoramento e logging são necessárias para garantir a visibilidade e a integridade do sistema.

Ferramentas e tecnologias comuns na arquitetura baseada em APIs

- **Swagger/OpenAPI:** ferramentas para documentar e gerenciar APIs, facilitando a integração e a manutenção.
- **Kong, Apigee:** plataformas de gestão de API gateways que ajudam a roteirizar solicitações, aplicar políticas de segurança e monitorar o tráfego de API.
- **OAuth, JWT:** protocolos de autenticação e autorização para proteger as APIs e garantir que apenas usuários e serviços autorizados possam acessar os dados.

Exemplo prático

Imagine um sistema de gerenciamento de saúde, onde há serviços separados para agendamento de consultas, gerenciamento de pacientes, processamento de

Quando usar microserviços e monolitos

pagamentos e envio de notificações. Cada um desses serviços expõe uma API que permite que o portal do paciente, o aplicativo móvel e o sistema administrativo interajam com eles.

Estrutura

Serviço de agendamento de consultas: expondo uma API para criar, modificar e cancelar consultas.

Endpoint REST: <https://api.saude.com/agendamentos>

Exemplo de solicitação:

```
{
  "pacienteId": 789,
  "medicoId": 456,
  "data": "2024-07-10T10:00:00Z"
}
```

Código-fonte 5 – Exemplo 5
Fonte: elaborado pelo autor (2024)

Exemplo de resposta:

```
{
  "agendamentoId": 123,
  "status": "confirmado"
}
```

Código-fonte 6 – Exemplo 6
Fonte: elaborado pelo autor (2024)

Serviço de gerenciamento de pacientes: expondo uma API para gerenciar informações dos pacientes.

Endpoint GraphQL: <https://api.saude.com/graphql>

Exemplo de consulta:

```
query {
  paciente(id: 789) {
    nome
    idade
  }
}
```

Quando usar microserviços e monolitos

```
historicoMedico {  
    doenca  
    tratamento  
}  
  
}
```

Código-fonte 7 – Exemplo 7
Fonte: elaborado pelo autor (2024)

Exemplo de resposta:

```
{  
  "data": {  
    "paciente": {  
      "nome": "Ana Silva",  
      "idade": 30,  
      "historicoMedico": [  
        {  
          "doenca": "Hipertensão",  
          "tratamento": "Medicação diária"  
        }  
      ]  
    }  
  }  
}
```

Código-fonte 8 – Exemplo 8
Fonte: elaborado pelo autor (2024)

Serviço de processamento de pagamentos: utilizando gRPC para transações seguras e rápidas.

Endpoint gRPC: <https://api.saude.com/pagamentos>

Exemplo de mensagem Protobuf:

```
message PagamentoRequest {  
    int32 pacienteId = 1;  
    double valor = 2;
```

Quando usar microserviços e monolitos

```
    string metodoPagamento = 3;
}

message PagamentoResponse {
    bool sucesso = 1;
    string mensagem = 2;
}
```

Código-fonte 9 – Exemplo 9
Fonte: elaborado pelo autor (2024)

Exemplo de solicitação e resposta: a comunicação ocorre por meio de streams gRPC, onde o cliente envia uma solicitação de pagamento e o servidor responde com o status da transação.

Serviço de notificações: envia notificações via API quando eventos ocorrem, como confirmações de consulta ou lembretes de pagamento.

Endpoint REST: <https://api.saude.com/notificacoes>

Exemplo de solicitação:

```
{
  "pacienteId": 789,
  "tipo": "lembrete",
  "mensagem": "Sua consulta está confirmada para 10 de julho às 10h."
}
```

Código-fonte 20 – Exemplo 10
Fonte: elaborado pelo autor (2024)

Exemplo de resposta:

```
{
  "notificacaoId": 321,
  "status": "enviado"
}
```

Código-fonte 31 – Exemplo 11
Fonte: elaborado pelo autor (2024)

Um API Gateway centraliza as solicitações, roteando-as para os serviços apropriados e aplicando políticas de segurança, como autenticação e autorização.

Modelos de arquitetura

Arquitetura orientada a eventos

Esta arquitetura utiliza eventos para comunicar e sincronizar os diferentes serviços. Os serviços publicam eventos quando algo relevante acontece, e outros serviços podem reagir a esses eventos. Essa abordagem promove a desacoplagem, permitindo maior flexibilidade e resiliência, especialmente em sistemas complexos e dinâmicos.

Ela é particularmente útil em sistemas onde as operações assíncronas são frequentes e onde a latência deve ser minimizada. Quando um evento significativo ocorre, como a criação de um novo pedido em um sistema de e-commerce, um evento é publicado em um barramento de mensagens. Outros serviços, como o de pagamento e o de envio, podem então consumir esse evento e realizar as ações necessárias de forma independente.

Estrutura da arquitetura orientada a eventos

A estrutura de uma arquitetura orientada a eventos inclui os seguintes componentes principais:

- **Eventos:** representam mudanças de estado ou ações que ocorrem dentro do sistema. Um evento pode ser, por exemplo, a criação de um novo pedido, o processamento de um pagamento, a atualização do status de envio etc
- **Produtores de eventos:** são serviços que publicam eventos quando algo relevante acontece. Cada evento é enviado para um barramento de mensagens onde pode ser consumido por outros serviços.
- **Consumidores de eventos:** são serviços que reagem aos eventos publicados. Eles consomem eventos do barramento de mensagens e realizam as ações necessárias, como atualizar dados, enviar notificações ou iniciar processos adicionais.

- **Barramento de mensagens:** atua como intermediário entre produtores e consumidores de eventos. Ferramentas como Apache Kafka e RabbitMQ são comumente utilizadas para gerenciar o fluxo de eventos e garantir a entrega e a ordem dos eventos.

Benefícios da arquitetura orientada a eventos

Promove a **desacoplagem** entre os serviços, aumentando a flexibilidade e a resiliência do sistema. Os serviços podem operar de forma independente, reagindo a eventos conforme necessário.

Escalabilidade e performance são ótimos benefícios desta arquitetura, que facilita a implementação de funcionalidades assíncronas e a escalabilidade de serviços individualmente. Cada serviço pode ser escalado conforme necessário para processar eventos de forma eficiente

Melhora a **resiliência** do sistema, permitindo que serviços falhem de forma independente sem afetar todo o sistema. Se um serviço falhar, outros serviços podem continuar a operar e processar eventos quando o serviço falho for restaurado.

Desafios da arquitetura orientada a eventos

A gestão de eventos pode se tornar complexa, exigindo ferramentas e práticas adequadas para garantir a consistência e a ordem dos eventos. É importante garantir que todos os eventos sejam entregues e processados corretamente.

Dificuldade em monitorar e depurar problemas devido à natureza assíncrona das interações. Ferramentas avançadas de monitoramento e logging são necessárias para garantir a visibilidade e a integridade do sistema.

Garantir a **consistência dos dados** em um sistema distribuído pode ser desafiador. Estratégias como eventual consistency e sagas são usadas para garantir a integridade dos dados em operações distribuídas.

Ferramentas e tecnologias comuns na arquitetura orientada a eventos

- **Apache Kafka:** uma plataforma de streaming distribuída utilizada para construir pipelines de dados em tempo real e aplicativos de streaming. Kafka é

Quando usar microserviços e monolitos

conhecido por sua alta taxa de transferência, baixa latência e durabilidade dos dados.

- **RabbitMQ:** um sistema de mensageria que facilita o gerenciamento de filas e a entrega de mensagens entre serviços. É amplamente utilizado por sua simplicidade e flexibilidade.
- **Event Sourcing:** um padrão onde o estado de uma aplicação é armazenado como uma sequência de eventos. Cada mudança de estado é registrada como um evento, o que facilita a reconstrução do estado da aplicação a qualquer momento.
- **CQRS (Command Query Responsibility Segregation):** um padrão que separa os modelos de leitura e escrita para melhorar o desempenho e a escalabilidade. Em CQRS, comandos modificam o estado e consultas leem o estado, permitindo otimizações específicas para cada operação.
- **ElasticSearch e Kibana:** ferramentas para monitoramento e análise de logs, fornecendo visibilidade sobre o fluxo de eventos e ajudando a depurar problemas.

Exemplo prático: considere um sistema de gerenciamento de pedidos em um e-commerce. Quando um cliente faz um pedido, o serviço de pedidos publica um evento "Pedido Criado" no barramento de mensagens. O serviço de pagamento consome esse evento e processa o pagamento, publicando um evento "Pagamento Processado". O serviço de envio consome o evento de pagamento e inicia o processo de envio, publicando um evento "Envio Iniciado". Cada serviço opera de forma independente, mas coordena ações por meio de eventos, garantindo um fluxo eficiente e desacoplado de operações.

Estrutura

Serviço de pedidos: publica eventos quando um novo pedido é criado.

Evento publicado: **PedidoCriado**

Exemplo de mensagem:

```
{  
  "pedidoId": 123,
```


Quando usar microserviços e monolitos

```
{
  "clienteId": 789,
  "valorTotal": 299.99,
  "itens": [
    {"produtoId": 456, "quantidade": 2},
    {"produtoId": 789, "quantidade": 1}
  ]
}
```

Código-fonte 42 – Exemplo 12
Fonte: elaborado pelo autor (2024)

Serviço de pagamentos: consome eventos de criação de pedidos e processa o pagamento.

Evento consumido: **PedidoCriado**

Evento publicado: **PagamentoProcessado**

Exemplo de mensagem:

```
{
  "pedidoId": 123,
  "statusPagamento": "aprovado",
  "dataPagamento": "2024-07-10T11:00:00Z"
}
```

Código-fonte 53 – Exemplo 13
Fonte: elaborado pelo autor (2024)

Serviço de envio: consome eventos de pagamentos processados e inicia o envio do pedido.

Evento consumido: **PagamentoProcessado**

Evento publicado: **EnvioIniciado**

Exemplo de mensagem:

```
{
  "pedidoId": 123,
  "statusEnvio": "em processamento",
  "dataEnvio": "2024-07-10T12:00:00Z"
}
```

```
}
```

Código-fonte 64 – Exemplo 14
Fonte: elaborado pelo autor (2024)

Serviço de notificações: consome eventos e envia notificações para o cliente sobre o status do pedido.

Evento consumido: **EnvioIniciado**

Exemplo de mensagem:

```
{
  "clienteId": 789,
  "mensagem": "Seu pedido #123 está em processamento e será
enviado em breve."
}
```

Código-fonte 75 – Exemplo 15
Fonte: elaborado pelo autor (2024)

Detalhamento técnico

Barramento de mensagens: para garantir a entrega eficiente e ordenada dos eventos entre os serviços, Kafka facilita a criação de pipelines de dados em tempo real e permite o processamento de grandes volumes de eventos com baixa latência.

Produtores de eventos

Serviço de pedidos: implementa um produtor Kafka para publicar o evento **PedidoCriado** no tópico **pedidos**.

Código exemplo (Java com Spring Boot):

```
@Service
public class PedidoService {
    @Autowired
    private KafkaTemplate<String, PedidoEvento>
kafkaTemplate;

    public void criarPedido(Pedido pedido) {
        PedidoEvento evento = new PedidoEvento(pedido);
        kafkaTemplate.send("pedidos", evento);
    }
}
```

```
}
```

Código-fonte 86 – Exemplo 16
Fonte: elaborado pelo autor (2024)

1. Consumidores de Eventos:

- **Serviço de Pagamentos:** Implementa um consumidor Kafka para consumir eventos **PedidoCriado** e processar pagamentos.

Código exemplo (Java com Spring Boot):

```
@Service
public class PagamentoService {
    @KafkaListener(topics = "pedidos", groupId =
"pagamento-grupo")
    public void processarPedido(PedidoEvento evento) {
        // Processa pagamento
        PagamentoEvento pagamentoEvento = new
PagamentoEvento(evento.getPedidoId(), "aprovado");
        kafkaTemplate.send("pagamentos", pagamentoEvento);
    }
}
```

Código-fonte 97 – Exemplo 17
Fonte: elaborado pelo autor (2024)

MERCADO, CASES E TENDÊNCIAS

Uber e a Arquitetura Orientada a Eventos

A Uber é um exemplo de empresa que utiliza uma arquitetura orientada a eventos para gerenciar seu complexo sistema de transporte e logística. Ela enfrenta o desafio de coordenar milhões de corridas diárias em tempo real, o que exige um sistema altamente responsivo e resiliente.

Benefícios da arquitetura orientada a eventos para a Uber

Um dos benefícios é o seu desacoplamento, que promove a independência entre serviços, permitindo que o sistema continue a funcionar mesmo se partes dele falharem.

Esta arquitetura também facilita a **escalabilidade** horizontal, onde cada serviço pode ser dimensionado conforme necessário para atender à demanda.

A arquitetura orientada a eventos permite que a Uber **processe eventos em tempo real**, como solicitações de corrida, atualizações de localização de motoristas e status de pagamento.

Tendências emergentes

Serverless e Functions-as-a-Service (FaaS): uma tendência crescente é o uso de arquiteturas serverless, onde as pessoas desenvolvedoras escrevem funções que são executadas em resposta a eventos sem a necessidade de gerenciar servidores. Serviços como AWS Lambda, Google Cloud Functions e Azure Functions estão liderando essa inovação.

Arquiteturas híbridas: muitas empresas estão adotando uma abordagem híbrida, combinando microserviços com monolitos modulares para obter o melhor dos dois mundos. Isso permite uma transição suave e a possibilidade de evoluir gradualmente a arquitetura do sistema.

Observabilidade e monitoramento avançado: com a crescente complexidade dos sistemas de microserviços, a observabilidade tornou-se uma área crítica. Ferramentas avançadas de monitoramento, logging e tracing, como Prometheus,

Quando usar microsserviços e monolitos

Grafana, Jaeger e ELK Stack, são essenciais para manter a saúde e o desempenho do sistema.

Segurança Zero Trust: a segurança em ambientes de microsserviços está evoluindo para o modelo de Zero Trust, onde a confiança não é concedida implicitamente dentro da rede. Cada solicitação é autenticada e autorizada rigorosamente para minimizar riscos.

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos os dois principais modelos de arquitetura de microserviços: arquitetura baseada em APIs e arquitetura orientada a eventos. Aprendemos que a arquitetura baseada em APIs facilita a integração e a escalabilidade dos serviços por meio de APIs bem definidas e documentadas, permitindo a comunicação estruturada entre diferentes partes do sistema. Detalhamos os tipos de APIs, incluindo REST, GraphQL e gRPC, analisando suas características, benefícios e desafios, e vendo exemplos práticos de implementação.

Em seguida, discutimos a arquitetura orientada a eventos, que promove o desacoplamento e a resiliência do sistema por meio da comunicação assíncrona de eventos. Esta abordagem é especialmente útil para sistemas que requerem operações em tempo real e baixa latência. Exploramos componentes como produtores e consumidores de eventos, barramentos de mensagens e padrões como Event Sourcing e CQRS, juntamente com ferramentas como Apache Kafka e RabbitMQ.

Analizamos os benefícios e desafios de cada abordagem, e estudamos o caso prático da Uber, que adotou essas arquiteturas para resolver problemas de escalabilidade e resiliência. Também discutimos, de forma breve, tendências emergentes no mercado, como serverless, arquiteturas híbridas, observabilidade avançada e segurança Zero Trust, ilustrando como essas inovações continuam a moldar o desenvolvimento de sistemas modernos.

REFERÊNCIAS

FOWLER, M; LEWIS, J. Microservices: A definition of this new architectural term. Martin Fowler, 2014. Disponível em: <https://martinfowler.com/articles/microservices.html>. Acesso em: 10 jul. 2024.

INTRODUCING Domain-Oriented Microservice Architecture. Uber, 2020. Disponível em: <https://www.uber.com/en-BR/blog/microservice-architecture/>. Acesso em: 24 set. 2024.

NEWMAN, S. Building Microservices. 2nd ed. [s.l.]: O'Reilly Media, 2021.

RICHARDSON, C. **Microservices Patterns: With examples in Java**. [s.l.]: Manning Publications, 2018.

Quando usar microserviços e monolitos

PALAVRAS-CHAVE

Arquiteto, Arquitetura de Software, Microserviços, API, Rest, GraphQL, gRPC, Eventos.

EMSE

POSTECH